Documentação: Registro de Métricas de Consumo

Objetivo

Monitorar e consolidar os consumos de recursos da empresa (storage, CPU, memória, I/O, conexões, requests e custos) a partir de qualquer movimentação nas tabelas do schema sch_stock_track_hml.

◈ Estrutura

1. Tabela de métricas

Armazena todos os registros de consumo gerados pelos triggers.

```
CREATE TABLE sch_stock_track_hml.metricas_empresa ( id SERIAL PRIMARY KEY, id_empresa INT NOT NULL, data_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP, storage_utilizado NUMERIC DEFAULT 0, cpu_utilizada NUMERIC DEFAULT 0, memoria_utilizada NUMERIC DEFAULT 0, io_utilizado NUMERIC DEFAULT 0, conexoes_usadas INT DEFAULT 0, requests INT DEFAULT 0, preco_plano NUMERIC DEFAULT 0, preco_ultrapassado NUMERIC DEFAULT 0 );
```

2. **Função de registro**

Responsável por inserir os dados de consumo na tabela de métricas.

```
CREATE OR REPLACE FUNCTION sch_stock_track_hml.registrar_metricas() RETURNS TRIGGER AS $$ BEGIN INSERT INTO sch_stock_track_hml.metricas_empresa ( id_empresa, storage_utilizado, cpu_utilizada, memoria_utilizada, io_utilizado, conexoes_usadas, requests, preco_plano, preco_ultrapassado ) VALUES ( COALESCE(NEW.id_empresa, 0), COALESCE(NEW.storage_utilizado, 0), COALESCE(NEW.cpu_utilizada, 0), COALESCE(NEW.memoria_utilizada, 0), COALESCE(NEW.io_utilizado, 0), COALESCE(NEW.conexoes_usadas, 0), COALESCE(NEW.requests, 0), COALESCE(NEW.preco_plano, 0), COALESCE(NEW.preco_ultrapassado, 0) ); RETURN NEW; END; $$ LANGUAGE plpgsql;
```

3. **Triggers de movimentação**

Cada tabela do schema dispara a função após um INSERT.
Exemplo para algumas tabelas:

```
-- Área DROP TRIGGER IF EXISTS trg_metricas_area ON sch_stock_track_hml.area; CREATE TRIGGER trg_metricas_area AFTER INSERT ON sch_stock_track_hml.area FOR EACH ROW EXECUTE FUNCTION sch_stock_track_hml.registrar_metricas(); -- Cliente DROP TRIGGER IF EXISTS trg_metricas_cliente ON sch_stock_track_hml.cliente; CREATE TRIGGER trg_metricas_cliente AFTER INSERT ON sch_stock_track_hml.cliente FOR EACH ROW EXECUTE FUNCTION sch_stock_track_hml.registrar_metricas(); -- Venda DROP TRIGGER IF EXISTS trg_metricas_venda ON sch_stock_track_hml.venda; CREATE TRIGGER trg_metricas_venda AFTER INSERT ON sch_stock_track_hml.venda FOR EACH ROW EXECUTE FUNCTION sch_stock_track_hml.registrar_metricas();
```

Esse padrão se repete para todas as tabelas do schema (empresa, fornecedores, produtos, embalagem, etc.).

4. **View de consolidação mensal**

Agrupa os consumos por empresa e mês.

```
CREATE OR REPLACE VIEW sch_stock_track_hml.metricas_empresa_mensal AS SELECT id_empresa, DATE_TRUNC('month', data_registro) AS mes, SUM(storage_utilizado) AS total_storage, SUM(cpu_utilizada) AS total_cpu, SUM(memoria_utilizada) AS total_memoria, SUM(io_utilizado) AS total_io, SUM(conexoes_usadas) AS total_conexoes, SUM(requests) AS total_requests, MAX(preco_plano) AS preco_plano, SUM(preco_ultrapassado) AS preco_ultrapassado FROM sch_stock_track_hml.metricas_empresa GROUP BY id_empresa, DATE_TRUNC('month', data_registro) ORDER BY id_empresa, mes;
```

Como cada consumo é capturado

- **Storage utilizado** → movimentações de produtos, depósitos, prateleiras, embalagens.
- **CPU utilizada** → operações intensivas como vendas, cálculos de estoque.
- **Memória utilizada** → consultas e registros de clientes, fornecedores, usuários.
- **I/O utilizado** → movimentações físicas (entrada/saída de produtos).
- **Conexões usadas** → acessos de usuários, perfis e permissões.
- **Requests** → qualquer operação de inserção em tabelas de negócio.
- **Preço do plano / ultrapassado** → registros de empresa, planos e perfil de acesso.

🔷 Fluxo completo

1. **Movimentação** em qualquer tabela do schema.
2. **Trigger** dispara e chama registrar_metricas().
3. **Função** insere registro em metricas_empresa.
4. **View mensal** consolida os dados automaticamente.